

Python开发者必看！UV包管理器从0到1保姆级教程，彻底告别pip龟速和依赖地狱

目录

一、为什么我推荐你立刻上手UV？

二、UV安装：3种方式，总有一款适合你

2.1 最简单：用pip直接安装（推荐有Python环境的用户）

2.2 最纯净：PowerShell独立安装（推荐无Python环境或想隔离安装的用户）

2.3 安装后必做：检查是否成功

三、环境配置：让UV更好用的关键一步

3.1 修改默认路径（拯救你的C盘）

3.2 配置镜像源：下载速度从KB/s到MB/s的飞跃

四、UV核心功能：从初始化到依赖管理全流程

4.1 初始化项目：一行命令搞定项目骨架

4.2 虚拟环境：自动管理，告别手动激活

4.3 依赖管理：添加、更新、删除一条龙

五、实战案例：从零创建一个数据分析项目

5.1 项目目标

5.2 步骤详解

六、UV常用命令速查表（收藏备用）

七、常见问题解决：踩坑经验分享

7.1 安装后命令找不到：uv: command not found

7.2 下载依赖速度慢，一直卡住

7.3 虚拟环境创建失败：Python version 3.12 not found

7.4 环境变量修改后不生效

八、写在最后：UV值得你投入学习吗？

作为常年和Python 打交道的开发者，相信很多人都和我一样，对pip又爱又恨——爱它的简单直接，一行命令就能安装依赖；恨它装包时的“龟速”加载，以及虚拟环境管理的繁琐操作。直到半年前接触到UV，这个由Rust编写的现代Python包管理器，才真正让我体验到“丝滑”的Python开发流程。

如果你也受够了pip 的慢、virtualenv的繁琐、Poetry的复杂，那这篇UV从0到1保姆级教程，绝对值得你看到最后，小白也能轻松上手，看完直接提升开发效率！

一、为什么我推荐你立刻上手UV？

UV最核心的优势，就是“快”和“省心”，总结下来有4个关键点，每一个都戳中开发者的痛点：

- 🚀 **速度碾压pip**：底层用Rust重构，安装依赖的速度比pip快30-100倍！亲测安装pandas+numpy组合，pip需要2分15秒，UV仅用8秒就完成，再也不用盯着进度条发呆。
- 🤖 **自动管理虚拟环境**：不用记复杂的source activate（macOS/Linux）或Scripts\activate（Windows）命令，UV会自动识别项目环境，执行命令时自动切换，彻底解放双手。
- 🔒 **精准依赖锁定**：自动生成uv.lock文件，精确记录每一个依赖包的版本和依赖关系，完美解决“我这能跑，你那跑不了”的依赖地狱问题。
- 🌐 **全平台兼容**：Windows、macOS、Linux无缝支持，甚至能帮你管理多个Python版本，不用再为跨平台环境配置头疼。

一句话总结：UV = 更快的pip + 更智能的virtualenv + 更简洁的Poetry，集合了主流包管理器的所有优点，却没有它们的缺点。

Introduction

Getting started

Installation

First steps

Features

Getting help

Guides

Installing Python

Running scripts

Using tools

Working on projects

Publishing packages

Migration

Integrations

Concepts

Projects

Tools

Python versions

Configuration files

Package indexes

Resolution

Build backend

Authentication

Caching

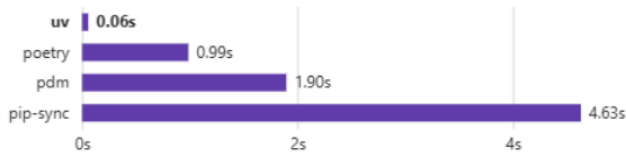
Preview features

The pip interface

Reference

UV

An extremely fast Python package and project manager, written in Rust.



Installing *Trio*'s dependencies with a warm cache.

Highlights

- 🚀 A single tool to replace `pip`, `pip-tools`, `pipx`, `poetry`, `pyenv`, `twine`, `virtualenv`, and more.
- ⚡ 10-100x faster than `pip`.
- 📁 Provides comprehensive project management, with a universal lockfile.
- 📄 Runs scripts, with support for inline dependency metadata.
- 🐍 Installs and manages Python versions.
- 🔧 Runs and installs tools published as Python packages.
- 📦 Includes a `pip-compatible` interface for a performance boost with a familiar CLI.
- li>• 🏠 Supports Cargo-style `workspaces` for scalable projects.
- 💾 Disk-space efficient, with a `global cache` for dependency deduplication.
- 📦 Installable without Rust or Python via `curl` or `pip`.

二、UV安装：3种方式，总有一款适合你

UV的安装非常简单，根据自身环境选择对应的方式即可，全程无需复杂配置，新手也能一键搞定。

2.1 最简单：用pip直接安装（推荐有Python环境的用户）

如果你的电脑里已经安装了Python（3.8及以上版本），直接用pip安装最方便，一行命令就能完成：

```
pip install uv
```

AI写代码python运行

安装完成后，用以下命令验证是否成功，出现版本号就说明安装OK：

```
uv --version # 示例输出: uv 0.1.32 (a1b2c3d4 2026-04-01)
```

AI写代码python运行

```
PS C:\Windows\system32> pip install uv
Looking in indexes: http://mirrors.aliyun.com/pypi/simple/
Collecting uv
  Downloading http://mirrors.aliyun.com/pypi/packages/bf/c0/f56ddb1b2276405618e3d2522018c962c010fc71f97f385d01b7e1dcd8df/uv-0.8.8-py3-none-win_amd64.whl (20.2 MB)
    |#####| 20.2 MB 3.3 MB/s
Installing collected packages: uv
Successfully installed uv-0.8.8
PS C:\Windows\system32> uv --version
uv 0.8.8 (9a54754b0 2025-06-05)
```

小技巧：如果提示“pip不是内部命令”，检查Python是否添加到系统环境变量，或用 `python -m pip install uv` 替代执行。

2.2 最纯净：PowerShell 独立安装（推荐无Python环境或想隔离安装的用户）

如果你想从零开始，或不想依赖系统自带的Python，推荐用官方的PowerShell脚本安装，会自动配置环境变量，甚至帮你安装Python（如果本地没有的话）。

默认安装（自动添加到环境变量）：

以管理员身份打开PowerShell，执行以下命令：

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

AI写代码python运行

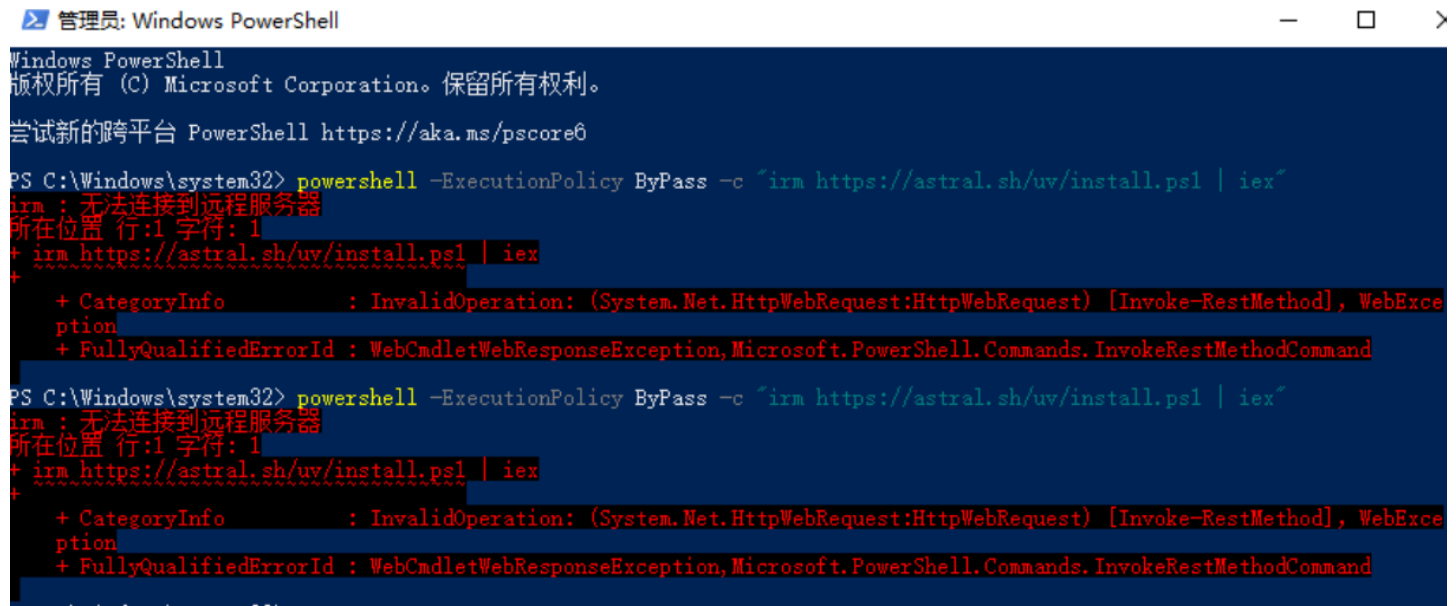
指定路径安装（避免C盘占用）：

如果想将UV安装到自定义目录（比如D:\Tools\uv），执行以下命令（替换路径即可）：

```
1 powershell -ExecutionPolicy ByPass -c {
2     $env:UV_INSTALL_DIR = "D:\Tools\uv"; # 这里替换成你的路径
3     irm https://astral.sh/uv/install.ps1 | iex
4 }
```

AI写代码python运行

如果遇到以下问题，可以采用2.1的方式安装



安装完成后，关闭PowerShell并重新打开，输入 `uv --version` 验证，出现版本信息即成功。

2.3 安装后必做：检查是否成功

不管用哪种方式安装，完成后一定要做这两步检查，避免后续踩坑：

1. 验证命令是否可用： `uv --version`（核心检查，必须能显示版本号）
2. 查看安装路径（可选）： `pip show uv`（仅pip安装方式可用）

```
Microsoft Windows [版本 10.0.19045.6093]
(c) Microsoft Corporation。保留所有权利。

C:\Users\hp>uv --version
uv 0.8.8 (9a54754b0 2025-08-08)

C:\Users\hp>pip show uv
Name: uv
Version: 0.8.8
Summary: An extremely fast Python package and project manager, written in Rust.
Home-page: https://pypi.org/project/uv/
Author:
Author-email: "Astral Software Inc." <hey@astral.sh>
License:
Location: e:\programdata\anaconda3\lib\site-packages
Requires:
```

如果提示“uv不是内部命令”，大概率是环境变量没配置好，重启电脑试试；若仍未解决，手动检查环境变量PATH中是否添加了UV的安装目录。

三、环境配置：让UV更好用的关键一步

刚装好的UV，默认会把缓存、工具和Python版本都放在C盘，时间长了会占用不少空间；而且国内访问PyPI官方源速度感人，必须配置**镜像源** 加速。这一步很重要，建议耐心看完！

3.1 修改默认路径（拯救你的C盘）

UV默认会把3个核心目录放在C盘用户目录下，我们可以通过环境变量，将它们修改到其他磁盘（比如E盘），具体如下：

目录类型	默认路径（C盘）	环境变量名
Python版本目录	C:\Users\用户名\.uv\python	UV_PYTHON_INSTALL_DIR
工具目录	C:\Users\用户名\.uv\tools	UV_TOOL_DIR
缓存目录	C:\Users\用户名\AppData\Local\uv\cache	UV_CACHE_DIR

```
E:\browser-use\web-ui>pip show uv
Name: uv
Version: 0.8.8
Summary: An extremely fast Python package and project manager, written in Rust.
Home-page: https://pypi.org/project/uv/
Author:
Author-email: "Astral Software Inc." <hey@astral.sh>
License:
Location: e:\programdata\anaconda3\lib\site-packages
Requires:
Required-by:

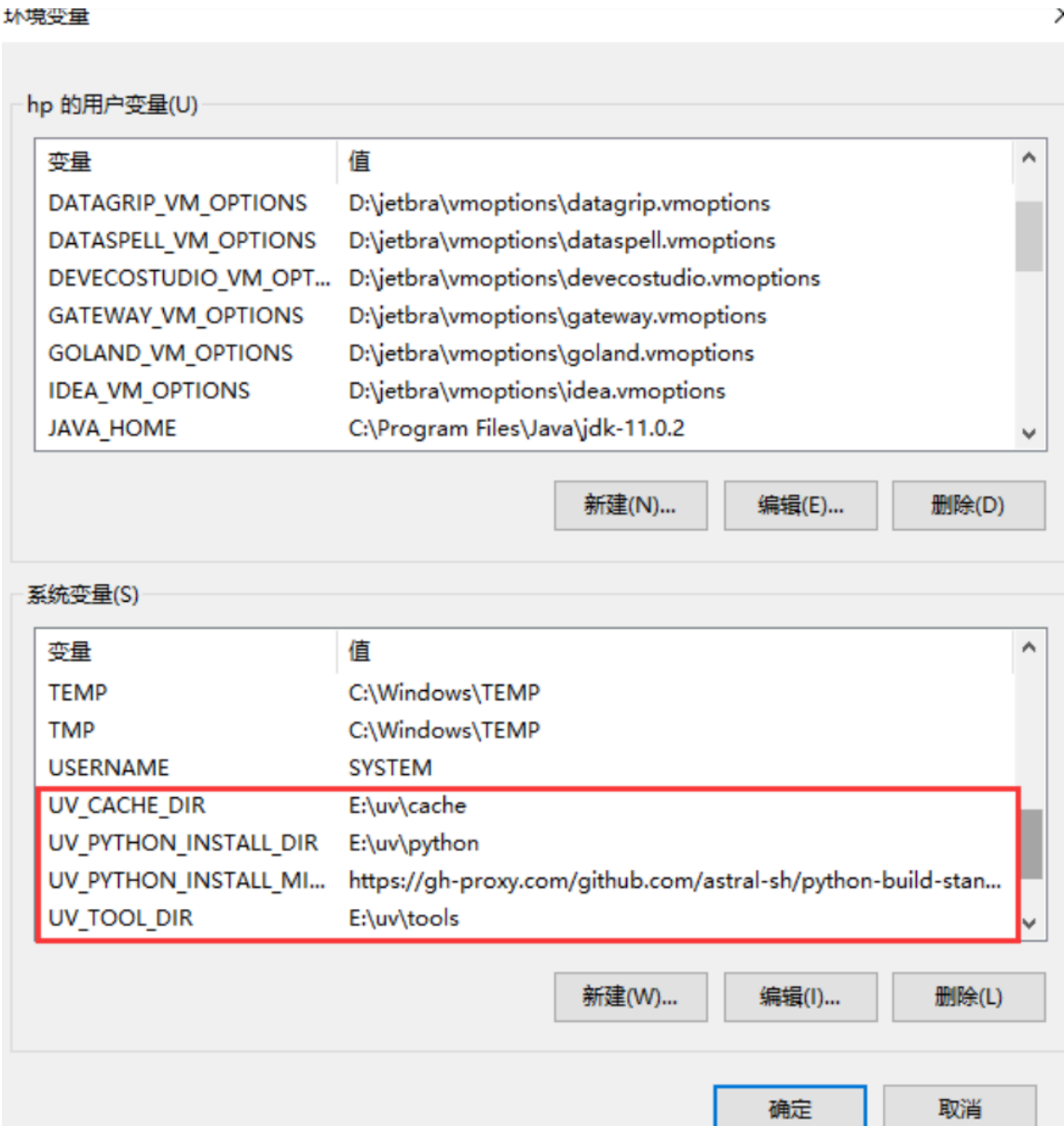
E:\browser-use\web-ui>uv python dir
C:\Users\hp\AppData\Roaming\uv\python

E:\browser-use\web-ui>uv tool dir
C:\Users\hp\AppData\Roaming\uv\tools

E:\browser-use\web-ui>uv cache dir
C:\Users\hp\AppData\Local\uv\cache
```

修改步骤：

- 1. 右键“此电脑”→“属性”→“高级系统设置”→“环境变量”；
- 2. 在“系统变量”中点击“新建”，分别添加上面3个环境变量，值为你想存放的路径（比如E:\UV\python、E:\UV\tools、E:\UV\cache）；
- 3. 额外添加一个环境变量 UV_PYTHON_INSTALL_MIRROR，值设为 <https://gh-proxy.com/github.com/astral-sh/python-build-standalone/releases/download/>（加快Python版本下载速度）；



4. 添加完成后，重启电脑生效。

重启后，用以下3个命令验证是否修改成功：

```
1 | uv python dir # 显示UV_PYTHON_DIR路径
2 | uv tool dir # 显示UV_TOOL_DIR路径
3 | uv cache dir # 显示UV_CACHE_DIR路径
```

AI写代码python运行

```
C:\Users\hp>uv --version
uv 0.8.8 (9a54754b0 2025-08-08)

C:\Users\hp>pip show uv
Name: uv
Version: 0.8.8
Summary: An extremely fast Python package and project manager, written in Rust.
Home-page: https://pypi.org/project/uv/
Author:
Author-email: "Astral Software Inc." <hey@astral.sh>
License:
Location: e:\programdata\anaconda3\lib\site-packages
Requires:
Required-by:

C:\Users\hp>uv python dir
C:\uv\python

C:\Users\hp>uv tool dir
C:\uv\tools

C:\Users\hp>uv cache dir
C:\uv\cache

C:\Users\hp>
```

3.2 配置镜像源：下载速度从KB/s到MB/s的飞跃

国内访问PyPI官方源 (<https://pypi.org/simple/>) 速度很慢，配置国内镜像源是刚需。推荐两种方式，优先选择第二种（项目级配置，更灵活）。

方式一：全局配置（所有项目生效）

1. 打开cmd命令行，输入 `md %UserProfile%\config\uv` 新建目录；
2. 再输入 `notepad %UserProfile%\config\uv\uv.toml`，创建uv.toml配置文件；



3. 在文件中添加以下内容，保存即可：

```
[[index]]
url = "https://pypi.tuna.tsinghua.edu.cn/simple"
default = true
```

方式二：项目级配置（推荐，不同项目可用不同源）

在项目根目录的pyproject.toml文件中添加（如果没有pyproject.toml，新建一个即可）：

```
[[index]]
url = "https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple/" # 清华源（稳定推荐）
default = true # 设置为默认源
```

小技巧：如果某个包在镜像源找不到，可以临时用官方源安装，命令：`uv add 包名 --index https://pypi.org/simple/`

四、UV核心功能：从初始化 到依赖管理全流程

UV最核心的功能是项目初始化、**虚拟环境** 管理和依赖管理，这三者无缝衔接，用起来比pip+virtualenv+pip-tools组合爽太多，全程命令简洁，无需复杂操作。

4.1 初始化项目：一行命令搞定项目骨架

新建Python项目，再也不用手动建文件夹、写requirements.txt了！UV的`uv init`命令会帮你生成完整的项目结构，一步到位。

基础用法：

- 1 | # 1. 新建文件夹并进入

```
2 | mkdir my_uv_project && cd my_uv_project 3 | # 2. 初始化项目（默认用系统Python版本）
4 | uv init
AI写代码python运行
```

执行后，目录下会生成以下文件（重点文件已标注作用）：

```
1 | my_uv_project/
2 | ├── .gitignore      # Git忽略文件（自动包含.venv等环境目录）
3 | ├── .python-version # 记录项目Python版本（类似pyenv）
4 | ├── main.py         # 示例入口脚本（可以删掉自己写）
5 | ├── pyproject.toml  # 项目配置文件（核心！记录依赖和设置）
6 | └── README.md       # 项目说明文档
```

AI写代码python运行

指定Python版本初始化（推荐）：

如果想指定项目用Python 3.12（或其他版本），初始化时直接加-p参数，UV会自动帮你安装该版本（如果本地没有）：

```
uv init my_uv_project -p 3.12 # 创建项目并指定Python 3.12
```

AI写代码python运行

这样生成的pyproject.toml中会明确写requires-python = ">=3.12"，后续依赖安装会自动匹配该版本。

4.2 虚拟环境：自动管理，告别手动激活

UV的虚拟环境管理是我最喜欢的功能——不用手动激活！UV会自动识别项目的.venv目录，执行命令时自动使用环境里的Python和依赖，彻底告别繁琐的激活命令。

创建虚拟环境：

```
1 | # 在当前项目创建虚拟环境，指定Python版本（推荐）
2 | uv venv --python 3.12 .venv # .venv是环境目录名，建议固定用这个
3 |
4 | # 或者初始化项目时直接创建（更高效）
5 | uv init my_project -p 3.12 # 会自动创建.venv目录
```

AI写代码python运行

执行后，项目根目录会多出.venv文件夹，里面就是独立的Python环境，不会影响系统Python和其他项目。

激活虚拟环境（可选，建议了解）：

虽然UV可以自动识别环境，但有时候需要手动在虚拟环境中执行命令（比如直接启动Python解释器），此时需要手动激活：

- Windows：. \.venv\Scripts\activate （激活后命令行前缀会显示(.venv)）
- macOS/Linux：source .venv/bin/activate

激活后，所有python、pip命令都会指向.venv里的版本，退出虚拟环境用deactivate命令即可。

最爽用法：UV自动激活环境执行脚本：

不用手动激活环境，直接用uv run执行脚本，UV会自动使用项目的虚拟环境，堪称懒人福音：

```
uv run main.py # 自动用.venv里的Python执行main.py
```

AI写代码python运行

删除虚拟环境：

直接删除项目根目录的.venv文件夹即可，简单粗暴，不会留下残留。

4.3 依赖管理：添加、更新、删除一条龙

UV的依赖管理比pip更智能，支持语义化版本约束，还能自动更新配置文件和锁文件，操作简洁高效。

添加依赖：

用uv add命令添加依赖，比pip install快得多，还会自动更新pyproject.toml和uv.lock文件：

```
1 | # 添加单个依赖（默认最新版）
2 | uv add pandas
3 |
4 | # 添加指定版本
5 | uv add requests==2.31.0
6 |
7 | # 添加开发环境依赖（仅开发时用，比如pytest）
```

```
8 | uv add pytest --dev # 会记录在pyproject.toml的[project.optional-dependencies.dev]里
```

AI写代码python运行

更新依赖：

```
1 | # 更新单个包到最新版
2 | uv add pandas --upgrade # 和uv update pandas效果一样
3 |
4 | # 更新所有依赖到兼容版本
5 | uv sync --upgrade
6 |
7 | # 更新锁文件（不修改pyproject.toml，只更新uv.lock）
8 | uv lock
```

AI写代码python运行

删除依赖：

```
uv remove pandas # 从依赖中删除pandas，同时更新pyproject.toml和uv.lock
```

AI写代码python运行

查看依赖树：

想知道项目依赖了哪些包，以及它们的依赖关系，用uv tree命令即可，清晰直观：

```
1 | uv tree # 显示所有依赖的树形结构
2 | uv tree pandas # 只显示pandas及其依赖
```

AI写代码python运行

输出示例（简化版）：

```
1 | pandas==2.2.2
2 | |— numpy>=1.26.0
3 | |— python-dateutil>=2.8.2
4 | |   |— six>=1.5
5 | |— pytz>=2020.1
```

AI写代码python运行

五、实战案例：从零创建一个数据分析项目

光说不练假把式，我们来从头到尾创建一个用UV管理的数据分析项目，体验完整流程，看完就能直接套用在自己的项目中。

5.1 项目目标

创建一个分析CSV数据的小项目，用pandas读取泰坦尼克号数据集，matplotlib绘制不同舱位的生存率柱状图，全程用UV管理环境和依赖。

5.2 步骤详解

步骤1：创建项目并初始化

```
1 | # 新建文件夹并进入
2 | mkdir uv_data_analysis && cd uv_data_analysis
3 |
4 | # 初始化项目，指定Python 3.12
5 | uv init -p 3.12
```

AI写代码python运行

此时目录下会生成pyproject.toml、.venv等基础文件，重点关注pyproject.toml（后续配置依赖和镜像源）。

步骤2：配置项目级镜像源

打开项目根目录的pyproject.toml文件，在末尾添加清华镜像源（确保下载速度）：

```
1 | [[index]]
2 | url = "https://mirrors.tuna.tsinghua.edu.cn/pypi/web/simple/"
3 | default = true
```

AI写代码python运行

步骤3：添加项目依赖

安装pandas（数据读取与分析）和matplotlib（绘图），用UV添加依赖，速度远超pip：


```
uv add pandas matplotlib
```

AI写代码python运行

执行后，UV会自动下载依赖，生成uv.lock文件，用清华源的话，10秒左右就能完成。

步骤4：编写分析代码

在项目根目录创建analyze.py文件，写入以下代码（简单易懂，注释清晰）：

```
1 | import pandas as pd
2 | import matplotlib.pyplot as plt
3 |
4 | # 读取数据（这里用pandas自带的示例数据）
5 | df = pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")
6 |
7 | # 简单分析：统计不同舱位的生存率
8 | survival_by_class = df.groupby("Pclass")["Survived"].mean()
9 |
10 | # 绘图
11 | survival_by_class.plot(kind="bar", title="Survival Rate by Passenger Class")
12 | plt.ylabel("Survival Rate")
13 | plt.xlabel("Passenger Class")
14 | plt.show()
```

AI写代码python运行



步骤5：运行脚本

不用激活虚拟环境，直接用UV运行脚本，自动识别项目环境：

```
uv run analyze.py
```

AI写代码python运行

执行后，会自动打开一个窗口，显示不同舱位的生存率柱状图，搞定！整个流程下来，没有繁琐的配置，全程丝滑。

六、UV常用命令速查表（收藏备用）

为了方便大家日常使用，整理了UV最常用的命令，按功能分类，一目了然，建议收藏，用到时直接查找：

功能分类	命令	说明	示例
项目初始化	uv init [项目名]	创建新项目，生成基础结构	uv init my_project -p 3.12
虚拟环境	uv venv --python <版本>	创建虚拟环境	uv venv --python 3.12 .venv
	uv run <脚本>	自动激活环境执行脚本	uv run main.py
	deactivate	退出虚拟环境	deactivate
	rmdir /s .venv （Windows）	删除虚拟环境	rmdir /s .venv
依赖管理	uv add <包名>	添加依赖，更新配置文件	uv add pandas --dev
	uv remove <包名>	删除依赖，更新配置文件	uv remove pandas
	uv sync	同步依赖（根据pyproject.toml安装）	uv sync --onlyprod
	uv lock	生成/更新锁文件	uv lock
	uv tree [包名]	查看依赖树	uv tree pandas
Python版本	uv python list	查看已安装Python版本	uv python list
	uv python install <版本>	安装指定Python版本	uv python install 3.11 3.12
	uv python pin <版本>	锁定项目Python版本	uv python pin 3.12.0
配置查看	uv python dir	查看Python版本存放目录	uv python dir
	uv cache dir	查看缓存目录	uv cache dir

七、常见问题解决：踩坑经验分享

我刚开始用UV时踩了不少坑，这里总结几个高频问题，帮你少走弯路，遇到问题直接对照解决即可。

7.1 安装后命令找不到：uv: command not found

原因：环境变量没配置好，或UV安装路径未加入系统PATH。

解决方法：

- 重启电脑（环境变量修改后，重启是最靠谱的生效方式）；
- 手动检查环境变量PATH，添加UV的安装目录（比如D:\Tools\uv\bin）；
- 如果是PowerShell安装，重新执行安装命令，注意查看输出是否有“添加到环境变量”的提示。

7.2 下载依赖速度慢，一直卡住

原因：未配置国内镜像源，或镜像源地址错误、失效。

解决方法：

- 检查uv.toml（全局配置）或pyproject.toml（项目级配置）中的镜像源URL是否正确，推荐使用清华源或阿里云源；
- 临时指定源安装：uv add 包名 --index https://mirrors.aliyun.com/pypi/simple/。

7.3 虚拟环境创建失败：Python version 3.12 not found

原因：本地没有安装指定的Python版本，且UV无法自动下载（可能是网络问题）。

解决方法：

- 先用uv python list查看本地已安装的Python版本；
- 手动安装指定版本：uv python install 3.12；
- 安装完成后，再创建虚拟环境：uv venv --python 3.12 .venv。

7.4 环境变量修改后不生效

原因：未重启电脑，或修改的是“用户变量”而非“系统变量”。

解决方法：

- 必须重启电脑（Windows环境变量修改后，重启才能完全生效）；
- 确认修改的是“系统变量”（所有用户生效），而非“用户变量”（仅当前用户生效）。

八、写在最后：UV值得你投入学习吗？

用了半年UV后，我已经把所有Python项目都迁移到UV管理了。如果你是以下这类开发者，UV绝对值得你花1小时学习——学会后节省的时间，远超你投入的学习成本。

- ✅ 经常创建新项目，受够了手动配置虚拟环境；
- ✅ 觉得pip安装依赖太慢，想提升开发效率；
- ✅ 需要精确控制依赖版本，避免“依赖地狱”；
- ✅ 讨厌复杂的配置，喜欢简洁、高效的工具。

UV可能不是完美的，但相比传统的pip+virtualenv组合，它的体验提升是革命性的——更快、更智能、更省心，让你把更多精力放在代码本身，而不是环境配置上。

最后，附上UV常用的官方资源，遇到问题可以随时查阅：

- UV官方文档（最权威，建议常备）：<https://docs.astral.sh/uv/>
- UV GitHub仓库（提issue或查看他人踩坑经验）：<https://github.com/astral-sh/uv>
- 国内常用镜像源：清华源、阿里云源（前面已详细介绍配置方法）

如果这篇教程对你有帮助，欢迎点赞收藏，有问题可以在评论区留言，我会尽量回复～祝大家用UV用得愉快，开发效率up up！